



Digital Integrated Circuit Design

Lecture-2: Review of Verilog HDL

Chih-Feng Wu (吳志峰), PhD.

*Department of Electronics Engineering, Chang Gung University,
Taoyuan, Taiwan (ROC)*

Mar. 6, 2026

Chih-Feng Wu



Outline

- ❑ **Verilog HDL**
 - **Overview**
 - **Dataflow Modeling**
 - **Behavior Modeling**

Chih-Feng Wu



Outline

- ❑ Verilog HDL
 - Overview
 - Dataflow Modeling
 - Behavior Modeling



Hardware Description Language

- ❑ HDL: Hardware Description Language
- ❑ Two commonly used HDLs
 - Verilog HDL (also called Verilog)
 - VHDL (Very high-speed integrated circuits HDL)
- ❑ Features of HDLs
 - Describe a design at a very abstract level
 - Finish the functional verification in the early design cycle
 - Design HDL code as same as computer programming



Verilog HDL

- ❑ A general-purpose, easy to learn, and easy to use HDL language
- ❑ Allowing different levels of abstraction in the same module
- ❑ Programming Language Interface (PLI)

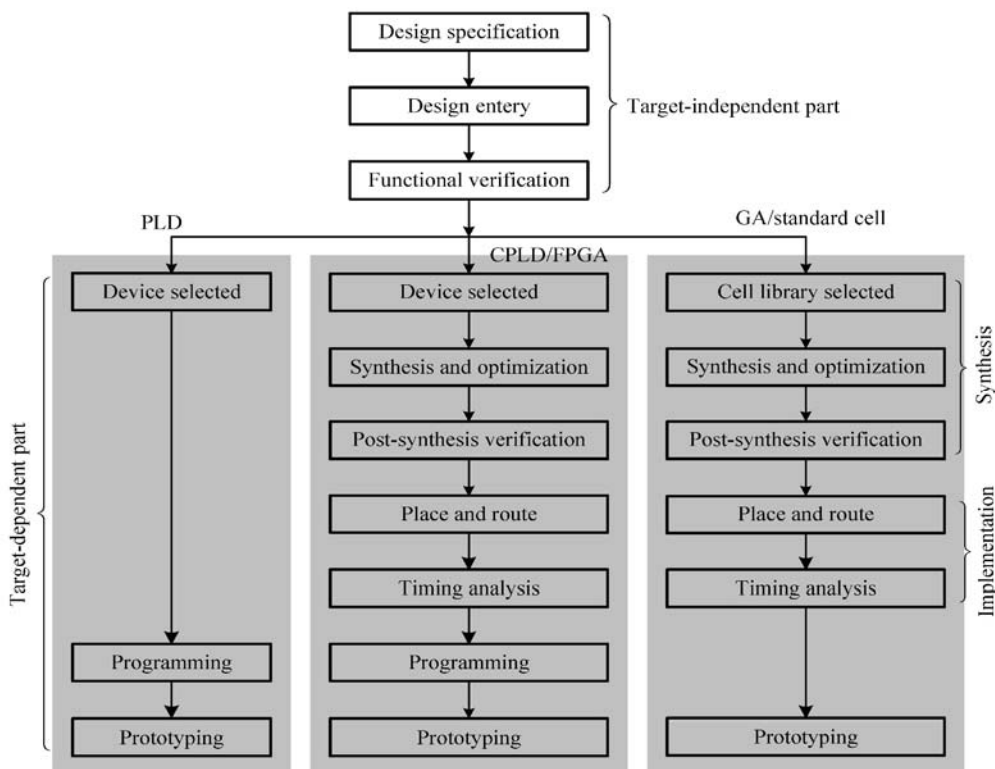


```
module simple_example_1; // a simple example to display a line of text.
  initial begin
    $display("Welcome to Verilog HDL World!\n");
  end
endmodule
```

```
module simple_example_2; // a simple example to display the sum of two numbers.
  integer a, b;
  initial begin
    a = 5; b = 8;
    $display("The sum of %2d and %2d is: %2d\n", a, b, a + b);
  end
endmodule
```

```
module nbit_adder(x, y, c_in, sum, c_out); /* I/O port declarations */
  parameter N = 4; // define the default value of N.
  input [N-1:0] x, y;
  input c_in;
  output [N-1:0] sum;
  output c_out;
  // specify the function of an n-bit adder using assign statement.
  assign #5 {c_out, sum} = x + y + c_in;
endmodule
```

HDL-Based Design Flow



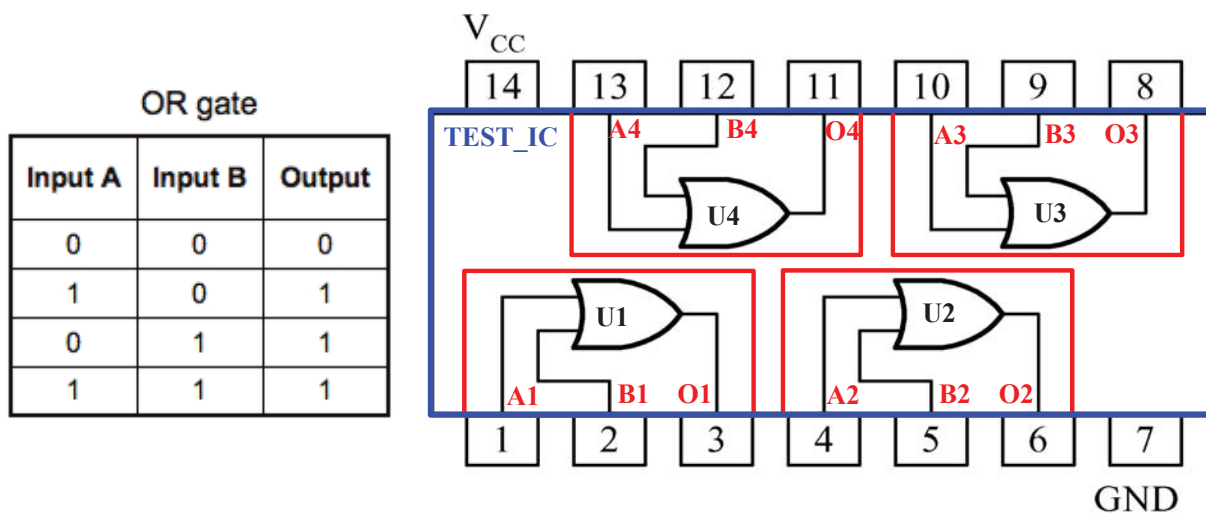
Chih-Feng Wu

7

Concept of Module

Module

- A core circuit (called internal or body)
- An interface (called I/O port)



Chih-Feng Wu

8



Concept of Module (cont'd)

```
module TEST_IC(I/O Port);  
    // O1, O2, O3, O4, A1, A2, A3, A4, B1, B2, B3, B4  
    input A1, A2, A3, A4;  
    input B1, B2, B3, B4;  
    output O1, O2, O3, O4;  
  
    OR U1(O1, A1, B1);  
    OR U2(O2, A2, B2);  
    OR U3(O3, A3, B3);  
    OR U4(O4, A4, B4);  
  
endmodule
```

Chih-Feng Wu

9



Modules – Verilog HDL Module

❑ **Module** --- The basic building block

`module` Module name

Port List, Port Declarations (if any)

Parameters (if any)

Declarations of *wires*, *regs*, and other variables

Instantiation of lower level modules or primitives

Dataflow statements (*assign*)

always and *initial* blocks. (all behavioral statements go into these blocks).

Tasks and functions.

`endmodule` statement

Chih-Feng Wu

10



Lexical Convention

- ❑ Almost the same lexical convention as C language
- ❑ Identifier:
 - A sequence of **alphanumeric characters**, **_**, and **\$**, for instance, counter four_bit_adder, _4b_adder, etc.
 - Verilog: **case-sensitive** language (two different identifiers: adder and Adder for different meaning)
 - **Keywords (reserved identifiers)**: **and**, **assign**, **begin**, **case**, **endcase**, **if**, **else**, **end**, **module**, **endmodule**, **input**, **output**, **wire**, **reg**, **posedge**, **negedge**, **always**, etc.
- ❑ **White space**: blank-space (\b), tab (\t), and new line (\n)
- ❑ **Comments**
 - // --- the remaining of the line
 - /* ...*/ --- what in between them
- ❑ **Sized number**: <size>'<**base format**><number>
 - 4'b1001
 - 16'habcd



Lexical Convention (cont'd)

- ❑ **Constant**: *integer, real and string*
 - decimal, hexadecimal, octal or binary
- ❑ **Unsigned number**:
 - '<base format><number>
 - 2009 // 32-bit decimal (default)
 - -123
 - `habc
- ❑ **Negative number**:
 - <size>'<base format><number>
 - -4'b1001
 - -16'habcd
- ❑ **Definition of constants**: **`define** (compiler directive) and parameter
 - **`define** BUS_WIDTH 8
 - parameter BUS_WIDTH 8



Lexical Convention (cont'd)

- ❑ Value set for four-value logic
 - 0 : logic 0, false condition
 - 1 : logic 1, true condition
 - z : high-impedance
 - x : unknown

- ❑ “_” and “?”
 - 16`b0101_1001_1110_0000
 - 8`b01??\_11?? // = 8`b01zz_11zz

- ❑ String: “Have a lovely day”

Chih-Feng Wu

13



Coding Style

- ❑ Lowercase letters
 - For all signal names, variable names, and port names
- ❑ Uppercase letters
 - For names of constants and user-defined types
- ❑ Meaningful names
 - For signals, ports, functions, and parameters

#Refer to RMM design

Chih-Feng Wu

14

Data Type

- ❑ **Nets:** any hardware connection points (physical connection)
- ❑ **Variables:** any data storage elements

	Nets	Variables
wire	supply0	reg
tri	supply1	integer
wand	tri0	real
wor	tri1	time
triand	triereg	realtime
trior	uwire	

Chih-Feng Wu

15

Nets

- ❑ For example (**wire** nets)

net_type [signed] [msb:lsb] net, net2, ..., netn;

wire a, b, c_in; // nets a, b, and c_in are 1-bit wires.

wire [7:0] data_a; // data_a is an 8-bit wire, the msb is bit 7.

wire [0:7] data_b; // data_b is an 8-bit wire, the msb is bit 0.

wire signed [7:0] data_c; // data_c is an 8-bit signed wire.

- ❑ Driven by
 - module port
 - continuous assignment
 - force ... release
 - Primitive

Chih-Feng Wu

16



Variables

☐ Assigned value only within

■ Procedural statement

■ Task

■ Function

☐ Cannot be used as

■ input and inout

☐ *reg* variable data type

reg [signed] [msb:lsb] reg1, reg2, ..., regN;

reg a, b, c_in; // reg a, b, and c_in are 1-bit reg variables.

reg [7:0] data_a; // data_a is an 8-bit reg, the msb is bit 7.

reg [0:7] data_b; // data_b is an 8-bit reg, the msb is bit 0.

reg signed [7:0] data_c; // data_c is an 8-bit signed reg.

Chih-Feng Wu

17



Module Modeling Style

☐ module = interface + internal

☐ Internal (body)

■ Structural style: a set of interconnected components

(a) Gate level and (b) Switch level

■ Dataflow style: specify the dataflow between registers and how to process the data

(a) A set of continuous assignment

(b) Behavioral or algorithmic style

(c) Mixed style (combining with user-defined modules)

☐ RTL (register transfer level)

■ synthesizable behavioral + dataflow construction

Chih-Feng Wu

18

Interface – Port Declaration

Types of ports

- input
- output
- inout

```
module nbit_adder(x, y, c_in, sum, c_out); /* I/O port declarations */
  parameter N = 4; // define the default value of N.
  input [N-1:0] x, y;
  input c_in;
  output [N-1:0] sum;
  output c_out;
  reg [N-1:0] sum;
  reg c_out;
  :
  :
endmodule
```

```
output reg [N-1:0] sum;
output reg c_out;
reg [N-1:0] sum;
reg c_out;
```

Interface – Port Declaration

```
module half_adder(x, y, s, c);
```

```
  input x, y;
  output s, c;
  // -- half adder body-- //
```

```
  // instantiate primitive gates
```

```
  xor xor1(s, x, y);
  and and1(c, x, y);
```

```
endmodule
```

Can only be connected by using positional association

Instance name is optional.

```
module full_adder(x, y, cin, s, cout);
```

```
  input x, y, cin;
  output s, cout;
  wire s1, c1, c2; // outputs of both half adders
  // -- full adder body-- //
```

```
  // instantiate the half adder
```

```
  half_adder ha_1(x, y, s1, c1);
```

```
  half_adder ha_2(.x(cin), .y(s1), .s(s), .c(c2));
```

```
  or(cout, c1, c2);
```

```
endmodule
```

Connecting by using positional association

Connecting by using named association

Instance name is necessary.

primitive

Port Connection Rule

- ❑ Named association
 - Specify the connection with listing their names and no need to follow the port order
- ❑ Positional association
 - Describe the connection with following their port order

Structural Modeling – Gate Level

```
// gate-level hierarchical description of
// 4-bit adder
// gate-level description of half adder
```

```
module half_adder (x, y, s, c);
  input x, y;
  output s, c;
```

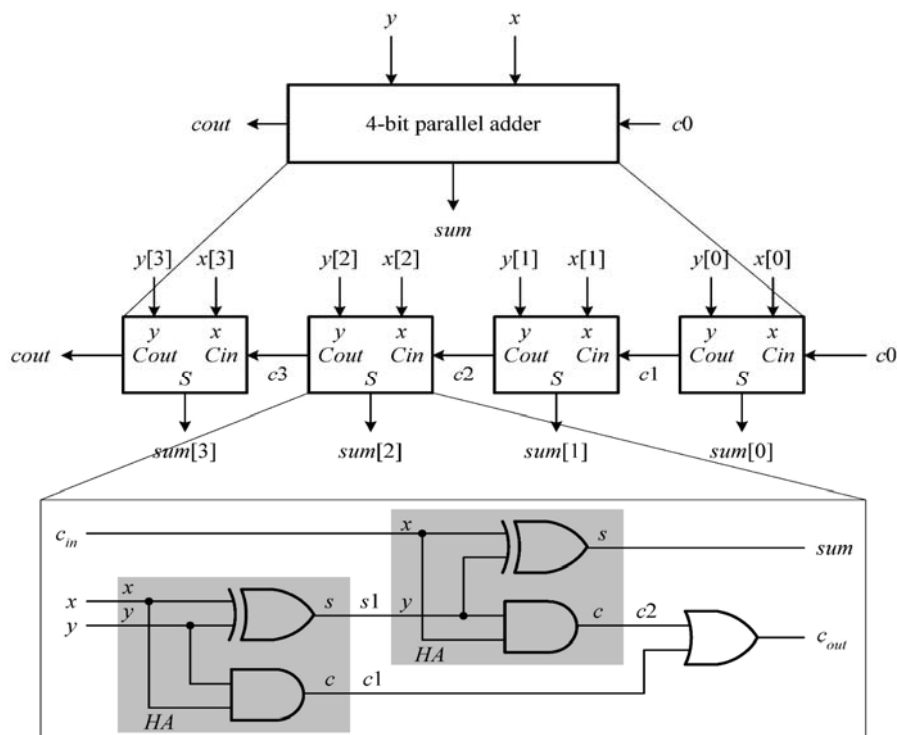
```
// half adder body
// instantiate primitive gates
xor DUT_XOR(s,x,y);
and DUT_AND(c,x,y);
endmodule
```

```
// gate-level description of full adder
```

```
module full_adder (x, y, cin, s, cout);
  input x, y, cin;
  output s, cout;
  wire s1, c1, c2;
  // outputs of both half adders
```

```
// full adder body
// instantiate the half adder
half_adder ha_1 (x, y, s1, c1);
half_adder ha_2 (cin, s1, s, c2);
or DUT_OR (cout, c1, c2);
endmodule
```

Hierarchical Design



Chih-Feng Wu

23

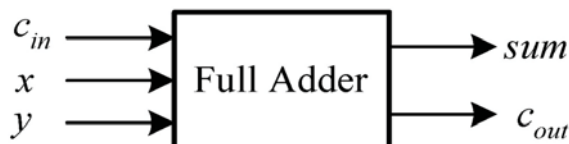
Dataflow Modeling

- Continuous assignment

`assign [delay] 1_value = expression;`

- For example: Full adder modeling in dataflow

```
module full_adder_dataflow(x, y, c_in, sum, c_out);
    // I/O port declarations
    input x, y, c_in;
    output sum, c_out;
    // specify the function of a full adder
    assign #5 {c_out, sum} = x + y + c_in;
endmodule
```



Chih-Feng Wu

24

Behavioral Modeling

□ Two procedural constructs

- **initial**: Only execute once and usually use to set up initial value
- **always**: Model **combinational** or **sequential logic**
- Both initial and always statements begin their execution at simulation time 0 concurrently.

```
module full_adder_behavioral(x, y, c_in, sum, c_out);
    // I/O port declarations
    input x, y, c_in;
    output sum, c_out;
    reg sum, c_out; // need to be declared as reg types
    // specify the function of a full adder
    always @(x, y, c_in) // always @(*) or always@(x or y or c_in)
        #5 {c_out, sum} = x + y + c_in;
endmodule
```

Chih-Feng Wu

25

Mixed-Style Modeling

```
module full_adder_mixed_style(x, y, c_in, s, c_out);
```

```
    // I/O port declarations
```

```
    input x, y, c_in;
```

```
    output s, c_out;
```

```
    reg c_out;
```

```
    wire s1, c1, c2;
```

```
    // structural modeling of HA 1
```

```
    xor xor_ha1 (s1, x, y);
```

```
    and and_ha1(c1, x, y);
```

```
    // dataflow modeling of HA 2
```

```
    assign s = c_in ^ s1;
```

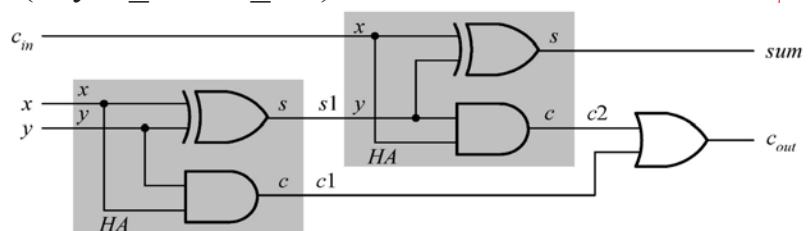
```
    assign c2 = c_in & s1;
```

```
    // behavioral modeling of output OR gate
```

```
    always @(c1, c2) // always @(*)
```

```
    c_out = c1 | c2;
```

```
endmodule
```

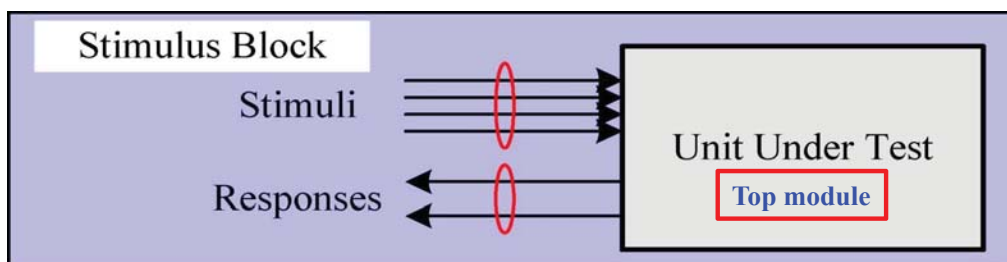


In structural modeling, only user-defined module is allowed in your Verilog code.

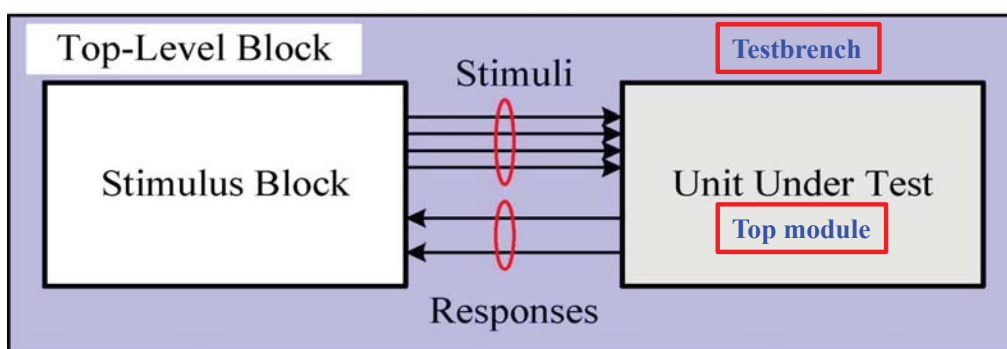
Chih-Feng Wu

26

Basic Simulation Construct



(a) Stimulus block at the top-level module.



(b) Stimulus block is considered as a separate module.

Chih-Feng Wu

27

Compiler Directive and System Task

- ❑ Compiler directive: ``timescale`
- ❑ Display system task: `$display` and `$monitor`
- ❑ Simulation time system functions: `$time` and `$realtime`
- ❑ Time scale compiler directive
 - ``timescale time_unit / time_precision`
 - `time_unit`: specify the unit of measure for times and delays
 - `time_precision`: describe how delay values rounded during simulation
 - `time_precision <= time_unit`

```
`timescale 10ns/1ns
```

```
#3.55 a= b+1; // 36 ns
#3.54 b=c=1; // 35 ns
```

```
`timescale 1ns/10ps
```

```
#3.55 a= b+1; // 3.55 ns
#3.54 b=c=1; // 3.54 ns
```

Chih-Feng Wu

28

System Task for Simulation

- ❑ **\$display**: display information only when it is called
 - `$display($time, "ns %d %d %h %h", x, y, c_in, {c_out, sum});`
- ❑ **\$monitor**: continuously monitor and display the values of any variables or expressions
 - `$monitor($realtime, "ns %d %d %h %h", x, y, c_in, {c_out, sum});`
- ❑ **\$monitotom**: enable \$monitor
- ❑ **\$monitotoff**: disable \$monitor
- ❑ **\$stop**: suspend the simulation
- ❑ **\$finish**: terminate the simulation

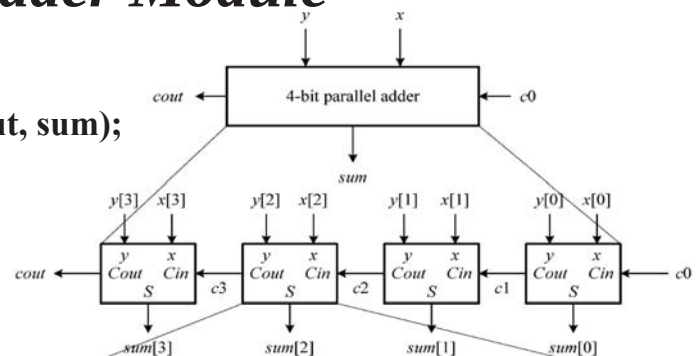
A 4-Bit Adder Module

// **gate-level description** of 4-bit adder
 module four_bit_adder (x, y, c_in, c_out, sum);

```

input [3:0] x, y;
input c_in;
output [3:0] sum;
output c_out;
wire c1, c2, c3; // intermediate carries
// -- four_bit adder body-- //
// instantiate the full adder
full_adder fa_1 (x[0], y[0], c_in, c1, sum[0]);
full_adder fa_2 (x[1], y[1], c1, c2, sum[1]);
full_adder fa_3 (x[2], y[2], c2, c3, sum[2]);
full_adder fa_4 (x[3], y[3], c3, c_out, sum[3]);
  
```

endmodule



```

// gate-level hierarchical description of
// 4-bit adder
// gate-level description of half adder
module half_adder (x, y, s, c);
input x, y;
output s, c;

// half adder body
// instantiate primitive gates
xor DUT_XOR(s,x,y);
and DUT_AND(c,x,y);
endmodule
  
```



Test Bench

```
`timescale 1 ns / 100 ps
module four_bit_adder_rtb; // internal signals declarations:

    reg [3:0] x;  reg [3:0] y;  reg c_in;  wire [3:0] sum;  wire c_out;
    // Unit Under Test instantiation and port map
    four_bit_adder DUT(.x(x), .y(y), .c_in(c_in),.sum(sum), .c_out(c_out));
    reg [7:0] i;
    initial
    begin // for use in post-map and post-par simulations.
        // $sdf_annotate ("four_bit_adder_map.sdf", four_bit_adder);
        // $sdf_annotate ("four_bit_adder_timesim.sdf", four_bit_adder);
    end
    // stimulus generation block
    for (i = 0; i <= 255; i = i + 1) begin
        x[3:0] = i[7:4];  y[3:0] = i[3:0];  c_in = 1'b0;
        #20 ;
    end
    initial #6000 $finish;
    initial // response monitoring block
        $monitor($realtime,"ns %h %h %h %h", x, y, c_in, {c_out, sum});

endmodule
```

Chih-Feng Wu

31



Simulation Result

realtime	x,	y,	c_in,	{c_out, sum}	realtime	x,	y,	c_in,	{c_out, sum}
0ns	0	0	0	00	# 320ns	1	0	0	01
# 20ns	0	1	0	01	# 340ns	1	1	0	02
# 40ns	0	2	0	02	# 360ns	1	2	0	03
# 60ns	0	3	0	03	# 380ns	1	3	0	04
# 80ns	0	4	0	04	# 400ns	1	4	0	05
# 100ns	0	5	0	05	# 420ns	1	5	0	06
# 120ns	0	6	0	06	# 440ns	1	6	0	07
# 140ns	0	7	0	07	# 460ns	1	7	0	08
# 160ns	0	8	0	08	# 480ns	1	8	0	09
# 180ns	0	9	0	09	# 500ns	1	9	0	0a
# 200ns	0	a	0	0a	# 520ns	1	a	0	0b
# 220ns	0	b	0	0b	# 540ns	1	b	0	0c
# 240ns	0	c	0	0c	# 560ns	1	c	0	0d
# 260ns	0	d	0	0d	# 580ns	1	d	0	0e
# 280ns	0	e	0	0e	# 600ns	1	e	0	0f
# 300ns	0	f	0	0f					

Chih-Feng Wu

32



Outline

☐ Verilog HDL

☐ Overview

☒ Dataflow Modeling

- Introduction
- Continuous assignment
- Expression
- Delay
- Operand
- Operator

☐ Behavior Modeling



Why Dataflow ?

☐ Any digital system

- Interconnecting registers
- Combinational logic circuits

☐ Features of Dataflow

- A **powerful** way to implement a design
- Used to create a gate-level netlist by logic **synthesis tools**
- A combination of **dataflow** and **behavioral** modeling for RTL (register transfer level)



Continuous Assignment

- ❑ Basic statement of dataflow modeling
 - Continuously drive a value onto a net, which means, always active
 - Only update values of net data type such as **wire**, *tiand* and so on
 - Appropriate to model the behavior of combinational logic
 - Keyword: **assign**

- ❑ Syntax

```
assign [delay] net_value = expression;  
assign [delay] net1_value = expr1, [delay] net2_value = expr2, ...,  
    [delay] netn_value = exprn;
```

- net_value: a scalar or vector net, or their concatenation
- For example,

```
assign #5 {c_out, sum[3:0]} = a[3:0] + b[3:0] + c_in;
```

Chih-Feng Wu

35



Continuous Assignment (cont'd)

- ❑ Net declaration assignment
 - Only one net declaration assignment per net

```
wire out; // regular continuous assignment  
assign out = in1 & in2;
```

```
wire out = in1 & in2; // net declaration assignment  
// (X) Not allowed in this class
```

- ❑ Implicit net declaration

```
wire in1, in2;  
assign out = in1 & in2; // (X) Not allowed in this class
```

- **out** is not declared as a wire, but an implicit wire declaration for out is done by the simulator.

Chih-Feng Wu

36

Expression

□ expression = operators + operands

- Operators: act on the operands to product desired results
- Like C programming language, have a rich set of operators including *arithmetic, shift, bitwise, case equality, reduction, logic, relational, equality and miscellaneous operators*
- Operands: any of the allowed data types
- Allowed operands: *constants, net, reg, integer, time, real, realtime, bit-select, part-select, array element and function calls*

Operators

Arithmetic	Bitwise	Reduction	Relational
+: add - : subtract * : multiply / : divide % : modulus ** : exponent	~ : NOT & : AND : OR ^ : XOR ~, ^~ : XNOR	& : AND : OR ~& : NAND ~ : NOR ^ : XOR ~, ^~ : XNOR	> : greater than < : less than >= : greater than or equal <= : less than or equal
Shift	case equality	Logical	Miscellaneous
<< : left shift >> : right shift <<< : arithmetic left shift >>> : arithmetic right shift	== : equality != : inequality Equality == : equality != : inequality	&& : AND : OR ! : NOT	{ , } : concatenation { c { } } : replication ? : conditional



Three Ways of Specifying Delay

- ☐ Regular assignment delay
- ☐ Net declaration assignment delay
- ☐ Net declaration delay

```
`define DLY (#10)

// regular assignment delay
wire out;
assign #10 out = in1 & in2;           // assign `DLY out = in1 & in2;

// net declaration assignment delay
wire #10 out = in1 & in2;           // (X) Not allowed in this class

// net delays
wire #10 out;                       // (X) Not allowed in this class
assign out = in1 & in2;
```

Chih-Feng Wu

39



Operand – Types of Constant

- ☐ integer:
 - Simple decimal form: -123, 12345
 - Base format notation: 16'habcd, 2009, 4'sb1001 // -7

[size]'[s/S][base_format] base_value
- ☐ real
 - Decimal notation: 1.5, .3 (illegal), 1294.872
 - Scientific notation: 15E12, 32E-6, 26.176_45_e-12 (ignore underscores)
- ☐ string
 - A sequence of characters enclosed by double quotes (“ ”)
 - Only specified in a single line
 - Each **character** represented as an **8-bit ASCII code**

Chih-Feng Wu

40

Example: String Manipulation

```
module string_test;
// internal signal declarations:
reg [8*25:1] str1;
reg [6*8:1] str2;
initial begin
  str1 = "Welcome to Digital World!";
  $display("%s\n", str1);
  str2 = "Hello!";
  $display("%s is stored as: %x\n", str2, str2); // ASCII code
end
endmodule
```

■ Useful special characters

\n \\ new line character
\t \\ tab character
\\ \\ character
\` \\ ` character
\ddd \\ a character specified in 3 octal digits

Operand – Data Type

- ❑ **Nets:** Any hardware connection points
 - **primitive**, continuous assignment (**assign**, force/release) or **module** ports
- ❑ **Variables:** Any data storage elements
 - **reg**, integer, time, real and realtime
- ❑ **reg variable**
 - Hold a value between assignments
 - May be used to model hardware registers
 - Example

```
reg a, b;
reg [7:0] data_a;
reg [0:7] data_b;
reg signed [7:0] d;
```

// (X) Not allowed in this class



Operand – integer Variable

- ❑ Contain integer values and use as a general-purpose variable for modeling high-level behavior
- ❑ Treated as a signed **reg** variable with the LSB being bit 0 and has at least 32 bits

- Example

```
integer i, j;  
integer data[7:0];
```



Operand – time Variable

- ❑ Used for storing and manipulating simulation time
- ❑ Used in conjunction with the **\$time** system task
- ❑ Only unsigned value and at least 64 bits, with the LSB being bit 0

- Example

```
time events;  
time current_time;
```



Operand – real and realtime Variables

- ❑ Cannot use range declaration
- ❑ Defaulted initial value to be zero (0.0)
 - Example

```
real events;           // declare a real variable
realtime current_time; // hold current time as real
```



Operand – Vectors

- ❑ A **vector** = multiple bit widths
 - [high:low] or [low:high]
 - The leftmost bit of MSB for [high:low]
 - Include **nets** and **reg** data types
- ❑ A **scalar** = 1-bit vector



Bit-Select and Part-Select

- ❑ **Bit-Select:** Extract a particular bit form a vector
 - `integer` and `time` can NOT be allowed to be vector but can be accessed by bit-select and part-select.
 - `real` and `realtime` are not allowed for bit-select and part-select.
- ❑ **Part-Select:** Extract a contiguous sequence of bits from a vector
 - Constant part-select:


```
reg [15:0] data_bus;
wire [7:0] a;
integer response_time;
::
:= data_bus[3:0];           // reg variable part-select
:= a[4:3];                 // net part-select
:= response_time [5:0];    // integer variable part-select
```
 - indexed part-select:


```
[<starting_bit>+:width]: data_bus[8+:8]    // select [15:8]
[<starting_bit>-:width]: data_bus[15-:4]    // select [15:12]
```

Chih-Feng Wu

47



Array and Memory Elements

- ❑ Allow all net and variable data types
- ❑ An array element can be a scalar or a vector.

```
wire a[3:0];           // a scalar wire array of 4-element
reg d[7:0];           // a scalar reg array of 8-element
wire [7:0] x[3:0];    // an 8-bit wire array of 4-element
reg [31:0] y[15:0];   // an 32-bit reg array of 16-element
integer states [3:0]; // an integer array of 4-element
time current[5:0];    // a time array of 6-element
```

Chih-Feng Wu

48

Memory

- ❑ Memory
 - model ROM, RAM, or register files
- ❑ Reference
 - a whole word
 - a portion of a word of memory

```

reg [7:0] mema [7:0];           // 1-D array of 8-bit vector
reg [7:0] memb [3:0][3:0];      // 2-D array of 8-bit vector
wire sum [7:0][3:0];           // 2-D array of scalar wire
// Bit- and part-select of memory array
mema[4][3]                     // the 3rd bit of the 4th element
mema[5][7:4]                   // the higher 4-bit of the 5th element
memb[3][1][1:0]                // the lower two bit of the 1st/3rd element
sum[5][0]                      // the 5th and 0th element
  
```

Bitwise Operator

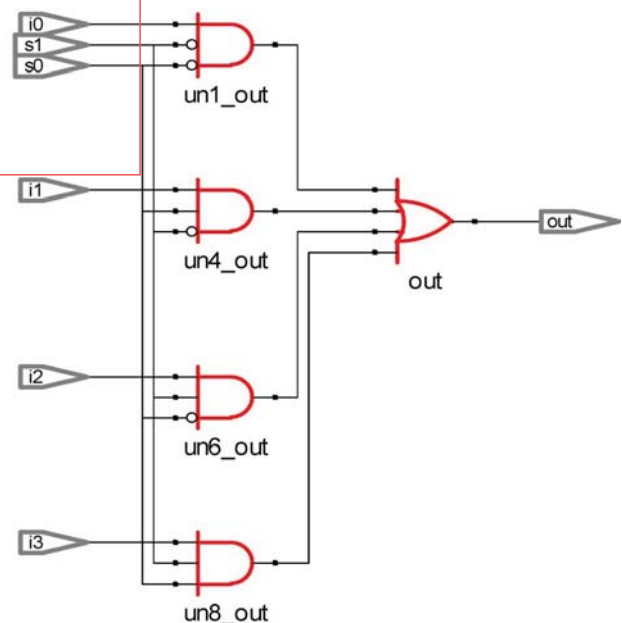
- ❑ A bit-by-bit operation
 - A z is treated as x
 - The shorter operand is zero-extended
- ❑ Bitwise operator

Symbol	Operation
~	Bitwise negation
&	Bitwise and
	Bitwise or
^	Bitwise exclusive or
~^, ^~	Bitwise exclusive nor

Example: A 4-to-1 MUX

// using basic and, or , not logic operators

```
assign out = (~s1 & ~s0 & i0) |
             (~s1 & s0 & i1) |
             (s1 & ~s0 & i2) |
             (s1 & s0 & i3);
```



Chih-Feng Wu

51

Arithmetic Operator

❑ Arithmetic operator

Symbol	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Exponent (power)
%	Modulus

❑ Negative number viewed as 2's complement

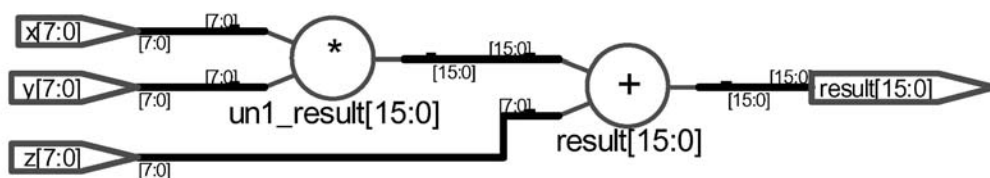
Chih-Feng Wu

52

Example

// an example to illustrate arithmetic operators

```
module multiplier_accumulator(x, y, z, result);
  input [7:0] x, y, z;
  output [15:0] result;
  assign result = x * y + z;
endmodule
```



A multiply and accumulate unit.

Chih-Feng Wu

53

Example

// an example to illustrate arithmetic operators

```
module arithmetic_operator(a, b, e, c, d);
```

```
  input [3:0] a, b;
```

```
  input [6:0] e;
```

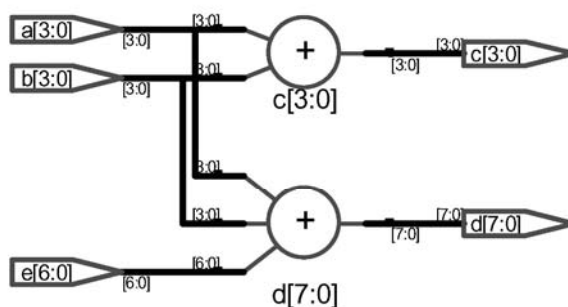
```
  output [4:0] c;
```

```
  output [7:0] d;
```

```
  assign c = a + b;
```

```
  assign d = a + b + e;
```

```
endmodule
```



The synthesized result of the arithmetic_operator module.

Chih-Feng Wu

54



Example: Mixed Signed and Unsigned Operands

// an example to illustrate arithmetic operators

```
module arithmetic_signed(a, b, e, c, d);
  input signed [3:0] a, b;
  input [6:0] e;
  output signed [4:0] c;
  output [7:0] d;

  assign c = a + b;
  assign d = a + b + e; // unsigned operator since of e (unsigned no.)

endmodule
```

Chih-Feng Wu

55



Concatenation/Replication Operator

❑ Operator

Symbol	Operation
{ , }	Concatenation
{const_expr{}}	Replication

❑ Concatenation operator

$y = \{a, b[0], c[1]\};$

$y = \{a, b[0], 4'b1\};$ *// 4'b1 = {1'b1, 1'b1, 1'b1, 1'b1}*

❑ Replication operator

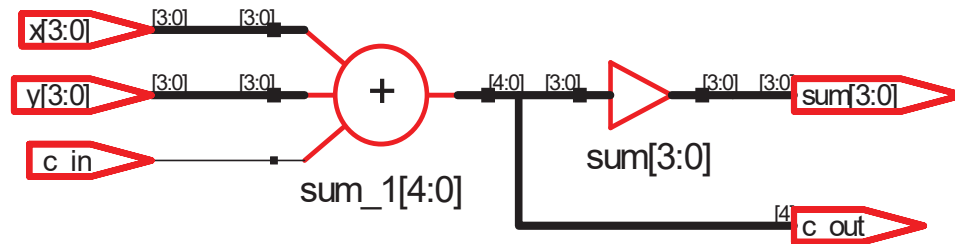
$y = \{a, \{4\{b[0]\}\}, c[1]\};$ *// {4{b[0]}} = {b[0], b[0], b[0], b[0]}*

Chih-Feng Wu

56

Example: A 4-Bit Full Adder

```
// specify the function of a 4-bit adder
assign {c_out, sum} = x + y + c_in;
```



Chih-Feng Wu

57

Reduction Operator

Operator

Symbol	Operation
&	Reduction and
~&	Reduction nand
	Reduction or
~	Reduction nor
^	Reduction exclusive or
~^, ^^	Reduction exclusive nor

Perform only on one vector operand

- Carry out a **bit-wise** operation
- Yield a **1-bit result**
- Work bit by bit **from right to left**

Chih-Feng Wu

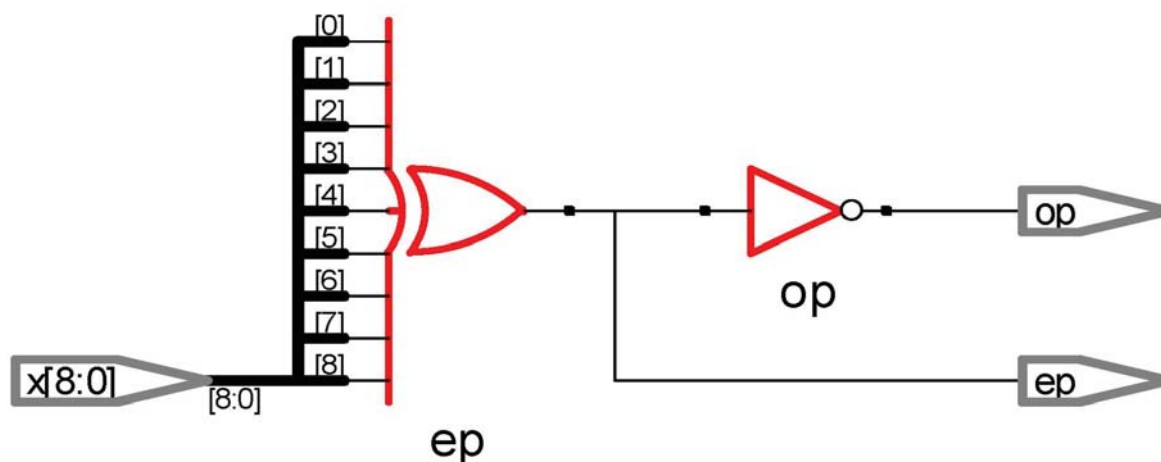
58

Example: A 9-Bit Parity Generator

// dataflow modeling using reduction operator

assign ep = ^x; // even parity generator

assign op = ~ep; // odd parity generator



Chih-Feng Wu

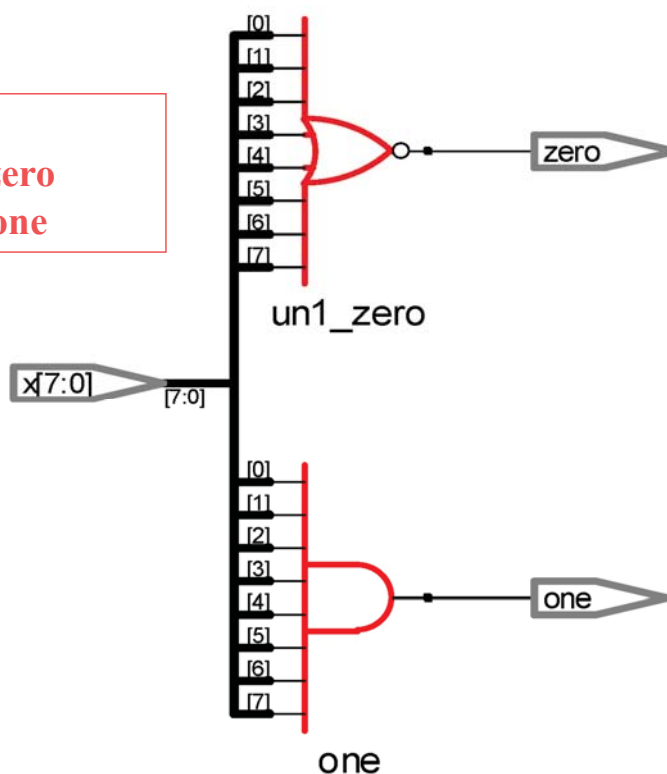
59

Example: An All-Bit-Zero/One Detector

// dataflow modeling

assign zero = ~(|x); // all-bit zero

assign one = &x; // all-bit one



Chih-Feng Wu

60

Logical Operator

Operator

Symbol	Operation
!	Logical negation
&&	Logical and
	Logical or

- Always evaluate on the logical values 0 or 1 and produce a 1-bit valued, 0, 1, or x.
- The result is **x** (a false condition) if any operand bit is **x** or **z**.
- A **non-zero vector** is treated as **1** for **vector operands**.

Example; c= 123 (integer) and d=0

reg a, b;

reg [7:0] c, d;

a= c || d; // a=1

b= c && d; // b=0

Chih-Feng Wu

61

Relational Operator

Operator

Symbol	Operation
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

- The result is 1 if the expression is true and 0 if the expression is false.
 - Return **x** if any operand bit is **x** or **z**.
- Unequal length between two operands
 - Unsigned: zero-extend to the small length of operand
 - Signed: signed-extended to the small length of operand

Chih-Feng Wu

62

Equality Operator

Operator

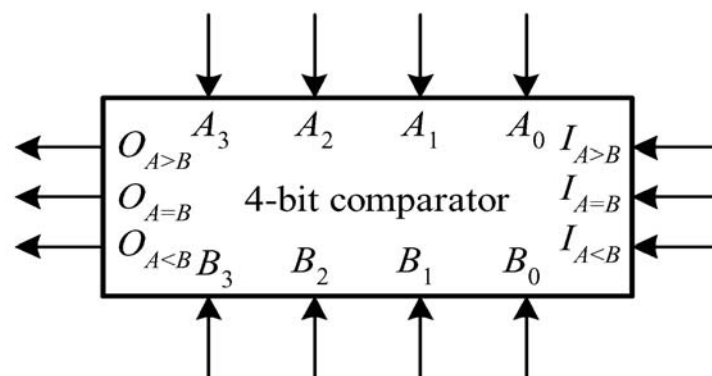
Symbol	Operation
==	Logical equality
!=	Logical inequality
===	Case equality
!==	Case inequality

- ❑ Compare the two operands bit by bit
 - Return 1 if the expression is true and 0 if the expression is false
 - The shorter operand is zero-extended
- ❑ Logic equality operators (==, !=)
 - Yield an unknown **x** if either operand has **x** or **z** in its bit.
- ❑ Case equality operators (===, !==)
 - Yield a 1 if the two operands match exactly
 - Get a 0 if the two operands do not match exactly

Chih-Feng Wu

63

Example: A 4-B Magnitude Comparator



// dataflow modeling using relation operators

```

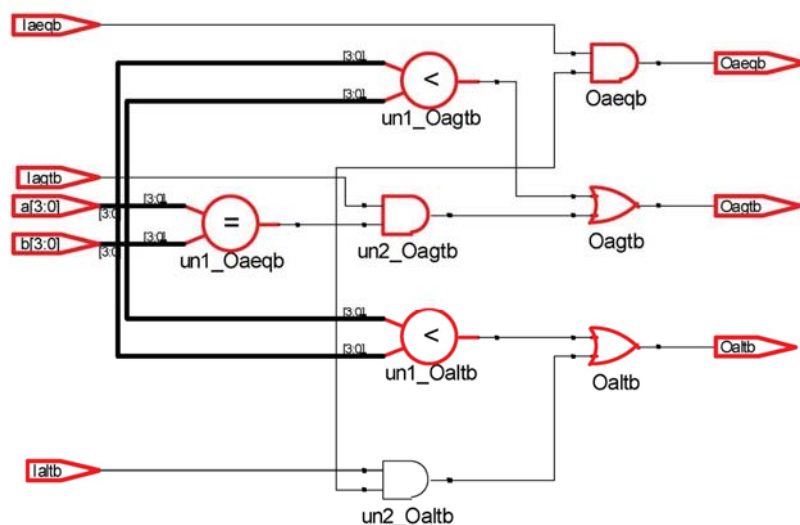
assign Oaeqb = (a == b) && (Iaeqb == 1);           // =
assign Oagtb = (a > b) || ((a == b) && (Iagtb == 1)); // >
assign Oaltb = (a < b) || ((a == b) && (Ialtb == 1)); // <

```

Chih-Feng Wu

64

Example: A 4-B Magnitude Comparator (cont'd)



The synthesized result of a cascadable 4-bit magnitude comparator.

Chih-Feng Wu

65

Shift Operator

Operator

Symbol	Operation
>>	Logical right shift
<<	Logical left shift
>>>	Arithmetic right shift
<<<	Arithmetic left shift

Logical shift operators

- Shift left: Vacant bit filled with zero
- Example: `start = 4'b0001; result = (start <<2);` → result = ?

Arithmetic shift operators

- Right shift: vacant bit filled with MSB (signed-bit)
- Example: `start = 4'b1000; result = (start >>>2);` → result = ?

Chih-Feng Wu

66



Example: Logical and Arithmetic Shifts

```
Module arithnmetic_shift (x, y, z);  
  input signed [3:0] x;  
  output [3:0] y;  
  output signed [3:0] z;  
  
  assign y = x >> 1;  
  assign z = x >>> 1;  
endmodule
```



Conditional Operator

- ❑ Usage: condition_expr ? true_statement : false_statement;
 - Pseudo code
if (condition_expr) true_statement
else false_statement
- ❑ For example

```
assign out = selection ? in_1 : in_0;
```

Example: A 4-to-1 MUX

// using the conditional operator

```
module mux4_to_1_cond (i0, i1, i2, i3, s1, s0, out);
```

```
  // port declarations from the I/O diagram
```

```
  input i0, i1, i2, i3;
```

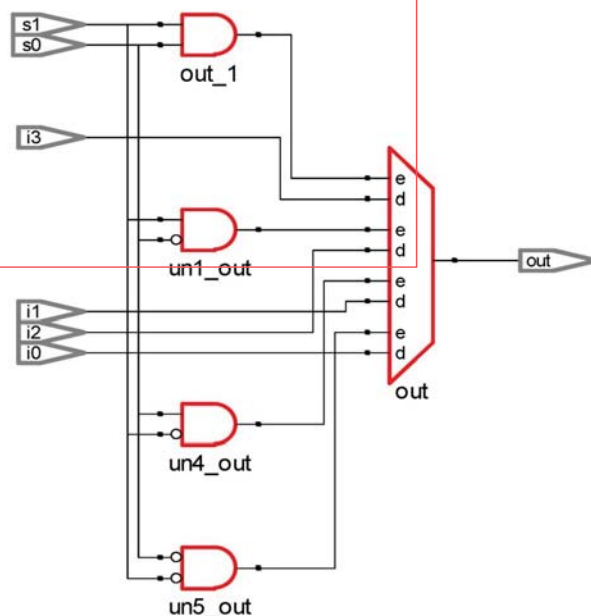
```
  input s1, s0;
```

```
  output out;
```

```
  // using conditional operator (?:)
```

```
  assign out = s1 ? (s0 ? i3 : i2) : (s0 ? i1 : i0);
```

```
endmodule
```



Chih-Feng Wu

69

Example

// Using the case equality and conditional operator

```
module mux41_equality(i0, i1, i2, i3, sel, out);
```

```
  // port declarations
```

```
  input i0, i1, i2, i3;
```

```
  input [1:0] sel;
```

```
  output out;
```

```
  // using case and conditional operators.
```

```
  assign out = (sel == 0) ? i0
```

```
               : (sel == 1) ? i1
```

```
               : (sel == 2) ? i2
```

```
               : (sel == 3) ? i3
```

```
               : 4'bz;
```

```
endmodule
```

// 4'bz → 4'b1111

Chih-Feng Wu

70



Outline

❑ Verilog HDL

- Overview
- Dataflow Modeling
- Behavior Modeling
 - Procedural construct
 - Procedural assignment
 - Timing control
 - Selection statement
 - Iterative (loop) statement
- Useful Model Technique



Procedural Construct

- ❑ Capture and describe the hardware feature using HDL
 - Continuity and parallelism
 - *initial* and *always* (Verilog HDL)
- ❑ *initial*
 - Used to **initialize variables** and **set values** into variables or nets
- ❑ *always*
 - Used to model the **continuous operation** required in the **hardware modules**
- ❑ All other **behavior** statements must be described within an *initial* or an *always* block.
- ❑ All *initial* and *always* statements execute **concurrently** with respect to each other, namely, their relative order in a module is not important.



An initial Block

- ❑ Execute exactly once during simulation
- ❑ Start at simulation **time 0**

```
reg x, y, z;
```

```
initial
```

```
  x = 1'b0; // single statement
```

```
initial begin // complex statement
```

```
  x = 1'b0; y = 1'b1; z = 1'b0;
```

```
  #10 x = 1'b1; y = 1'b1; z = 1'b1;
```

```
end
```

```
reg x, y, z;
```

```
initial
```

```
  #10 x = 1'b0; // finish at time #?
```

```
initial begin
```

```
  #10 y = 1'b1; // finish at time #?
```

```
  #20 z = 1'b1; // finish at time #?
```

```
end
```



Combined Variable Declaration and Initialization

```
reg clk;
initial clk=0;
```

```
reg clk=0;
```

```
module adder (input [3:0] x, y,
               input c_in,
               output reg [3:0] sum=0,
               output reg c_out=0);
```

```
module adder (x, y, c_in, sum, c_out);
```

```
  input [3:0] x, y;
```

```
  input c_in;
```

```
  output reg [3:0] sum=0;
```

```
  output reg c_out=0;
```

```
  :
```

```
endmodule
```

Not allow in this class.



An always block

- ❑ Start at simulation **time 0**
- ❑ Execute repeatedly the statements within it in a loop fashion during simulation

```
reg clk;                // clock generator

initial clk = 1'b0;      // initial clk=0

always #5 clk = ~clk;    // period = 10
```



Example

```
module mux41_behavioral(i0, i1, i2, i3, s1,
                        s0, out);
```

```
// port declarations
input i0, i1, i2, i3;
input s1, s0;
output reg out;
```

```
// using basic and, or, not logic operators.
```

```
always @(i0 or i1 or i2 or i3 or s1 or s0)
    out = (~s1 & ~s0 & i0) |
          (~s1 & s0 & i1) |
          ( s1 & ~s0 & i2) |
          ( s1 & s0 & i3);
```

```
endmodule
```

```
module four_bit_adder(x, y, c_in, sum,
                      c_out);
```

```
// I/O port declarations
input  [3:0] x, y; // declare as a 4-bit array
input  c_in;
output reg [3:0] sum; // declare as a 4-bit
                      // array
output reg c_out;
```

```
// specify the function of a 4-bit adder.
```

```
always @(x or y or c_in)
    {c_out, sum} = x + y + c_in;
```

```
endmodule
```



Outline

❑ Verilog HDL

- Overview
- Dataflow Modeling
- Behavior Modeling
 - Procedural construct
 - Procedural assignment
 - Timing control
 - Selection statement
 - Iterative (loop) statement
- Useful Model Technique



Procedural Assignment

❑ Two types of procedure assignment

- blocking using operator “=” to express **combinational logic** operation
- nonblocking using operator “<=” to express **sequential logic** operation

❑ Syntax

variable_lvalue = [*timing_control*] expression

[*timing_control*] *variable_lvalue* = expression

- *variable_lvalue*: reg, integer, time, real, realtime, an array (memory) element, a bit select, a part select or concatenation of any of the above

- *timing_control*: delay control or event control

variable_lvalue <= [*timing_control*] expression

[*timing_control*] *variable_lvalue* <= expression



Procedural Assignment (cont'd)

- ❑ Have several distinct features in comparison with continuous assignment
 - Do NOT use the keyword *assign*
 - Only update variable data types, *reg*, *integer*, *time*, *real* and *realtime*, or *array* (*memory*) elements
 - Only be used within *always* and *initial* blocks



Block Assignment

- ❑ Execute in the specified *sequential order*
- ❑ Use the “=” operator

```
// blocking assignments
initial begin
  x = #5 1'b0; // at time 5
  y = #3 1'b1; // at time 8
  z = #6 1'b0; // at time 14
end
```



Nonblocking Assignments

- ❑ Use the “<=” operator
- ❑ Execute without blocking the other statements in a sequential block
- ❑ Model several **concurrent** data transfers

```
reg x, y, z;  
// nonblocking assignments  
initial begin  
    x <= #5 1'b0; // x=0 at time 5  
    y <= #3 1'b1; // y=1 at time 3  
    z <= #6 1'b0; // z=0 at time 6  
end
```

- ❑ Suitable to model the **pipeline** and several **mutually exclusive** data transfers



Example: Right-Shift Register without Reset

```
// an example of right-shift register without reset.  
module shift_reg_4b(clk, din, qout);  
  
    input clk;  
    input din;  
    output reg [3:0] qout;  
  
    // the body of a 4-bit shift register  
    always @(posedge clk)  
        qout <= {din, qout[3:1]}; // right shift  
  
endmodule
```



Blocking vs. Nonblocking Assignment

❑ Race Problem

- Swap the contents of two registers

```
// using blocking assignment statements
always @(posedge clk) // race problem
begin
    x = y;
    y = x;                                ans: x= ___ and y= ___;
end

// using nonblocking assignment statements
always @(posedge clk) // no race problem
begin
    x <= y;
    y <= x;                                ans: x= ___ and y= ___;
end
```

Chih-Feng Wu

83



Blocking vs. Nonblocking assignment

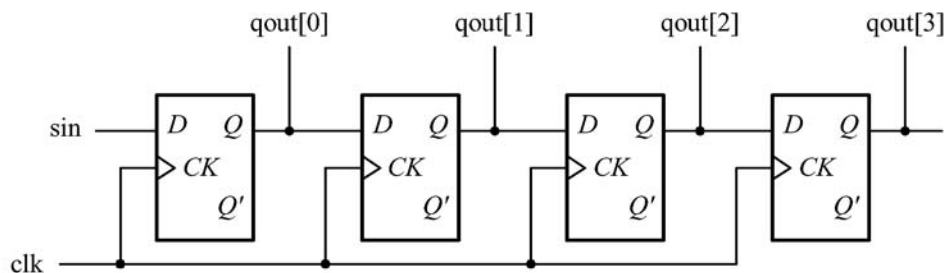
```
// using blocking assignment statements
always @(posedge clk) begin //without race problem
    temp_x = x;
    temp_y = y;

    y = temp_x;
    x = temp_y;
end                                ans: x= ___ and y= ___;
```

Chih-Feng Wu

84

Blocking Assignment for Shift-Register

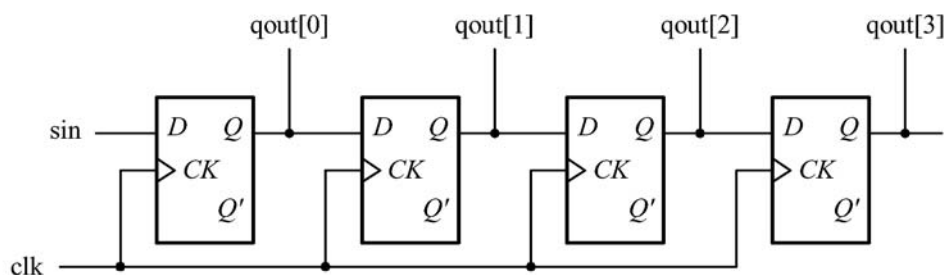


The basic structure of a 4-bit shift register.

```
// A 4-bit shift register
always @(posedge clk) begin
    qout[0] = sin;
    qout[1] = qout[0];
    qout[2] = qout[1];
    qout[3] = qout[2];
end
```

```
// A 4-bit shift register
always @(posedge clk) begin
    qout = {qout[2:0], sin};
end
```

Nonblocking Assignment for Shift-Register



The basic structure of a 4-bit shift register.

```
// A 4-bit shift register
always @(posedge clk) begin
    qout[0] <= sin;
    qout[1] <= qout[0];
    qout[2] <= qout[1];
    qout[3] <= qout[2];
end
```

```
// A 4-bit shift register
always @(posedge clk) begin
    qout <= {qout[2:0], sin};
end
```

Blocking vs. Nonblocking Assignments

- Suppose the initial value for `count = 1` and `finish = 0` before entering the always block

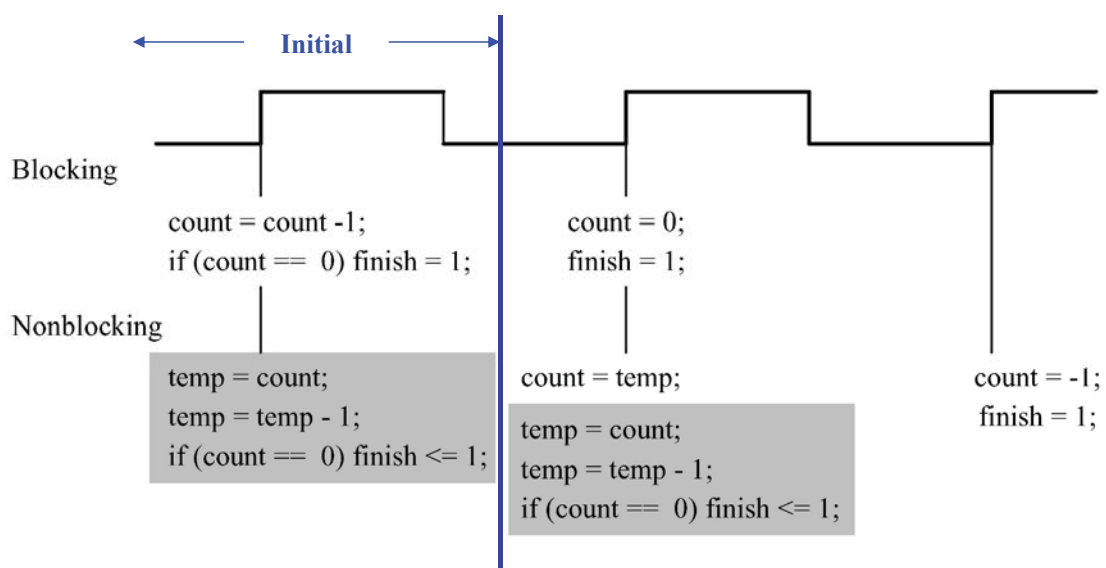
```
always @(posedge clk) begin: block_a
    count = count - 1;
    if (count == 0) finish = 1;
end
```

Result: `finish = ____`.
(Different from that of gate-level.)

```
always @(posedge clk) begin: block_b
    count <= count - 1;
    if (count == 0) finish <= 1;
end
```

Result: `finish = ____`.
(Same as that of gate-level.)

Timing Diagram for Blocking and Nonblocking



An example illustrates the differences between blocking and nonblocking assignments.



Outline

❑ Verilog HDL

- Overview
- Dataflow Modeling
- Behavior Modeling
 - Procedural construct
 - Procedural assignment
 - Timing control
 - Selection statement
 - Iterative (loop) statement
- Useful Model Technique



Delay Timing Control

- ❑ Two types of delay timing control
 - Regular delay control
 - Intra assignment delay control

Regular Delay Control

□ Regular delay control

- Defer the execution of the entire statement by a specified number of time unit
- Syntax: **#delay procedural_statement**

```
reg x, y, z;  
integer d, e;  
integer count;
```

```
#25      y <= ~x;           // at time 25  
#15      count <= count + 1; // at time 40  
#((d+e)/2) x = y;         // at time 40 + (d+e)/2  
#z       z = z + 1;        // at time 40 + (d+e)/2 + z
```

- No difference effect for both “<=” and “=”

Chih-Feng Wu

91

Intra-Assignment Delay Control

□ Intra-assignment delay control

- Defer the assignment to the left-hand-side variable by a special number of time unit, and meantime, the right-hand-side expression is evaluated.
- Syntax:
variable_lvalue = **#delay** expression
variable_lvalue <= **#delay** expression

```
y = #25 ~x;           // assign to y at time 25  
count = #15 count + 1; // assign to count at time 40
```

```
y <= #25 ~x;           // assign to y at time 25  
count <= #15 count + 1; // assign to count at time 15
```

- **#delay** can be a **constant** or an **expression**.

Chih-Feng Wu

92



Event Timing Control

- ❑ Procedural statement
 - Executed in synchronous with an event
- ❑ Two kinds of event control
 - Edge-triggered event control
 - Level-sensitive event control

Q: What is an event control?

Ans: For a procedural statement, defer the execution of the statement until the occurrence of the specified event.

Chih-Feng Wu

93



Edge-Triggered Event Control

- ❑ Edge-triggered event control
 - Defer its execution until specified transition of a given signal occurs
 - @ used to specify an **edge-triggered** event control
 - Syntax: @event procedural_statement
 - For example: @(enable) a=b; // controlled by enable
- ❑ Two types of edge-triggered event control
 - Negative (rising) transition, @(negedge clock): 1→x, z, 0; x or z →0;
 - Positive (falling) transition, @(posedge clock): 0→x, z, 1; x or z →1;
- ❑ Edge-triggered event control
 - Named event control
 - Event or control

```
always @(posedge clock) begin
    reg1 <= #25 in_1;
    reg2 <= @(negedge clock) in_2 ^ in_3;
    reg3 <= in_1;
end
```

Chih-Feng Wu

94

Name Event Control

❑ Named event: declare an identifier with an event

■ Syntax:

event identifier1, identifier2,, identifierN;

■ Event trigger

-> neme_event ;

```
event received_data;           // Step1: declare an event received_data

always @(posedge clock)        // Step2: trigger the event received_data
    if (last_byte) -> received_data;

always @(received_data)        // Step3: recognize the event
begin
    // put the required operation here
end
```

Event or Control

❑ Event or control:

■ In many applications, any one of multiple signals or events can trigger the execution of a procedural statement or a block of procedural statements.

■ Represented as a **sensitivity** list or **event** list

❑ Keyword: “**or**”, “**,**”, “*****” in a **sensitivity** list

■ (*) means **any** signals

```
// “or”
always @(a or b or c_in)
    {c_out, sum} = a+b+c_in;
```

```
// “,”
always @(a, b, c_in)
    {c_out, sum} = a+b+c_in;
```

```
// “*”
always @(*)
    {c_out, sum} = a+b+c_in;
```

```
// D-type FF with an asynchronous reset
always @(posedge clock or negedge reset_n)
    if (!reset_n) q <= 1'b0;
    else          q <= d;
```

Level-Sensitive Event Control

- ❑ Level-sensitive event control
 - The execution of a procedural statement is delayed until a condition is true.
 - Keyword: **wait**
 - Syntax: **wait (condition) procedural_statement**

always

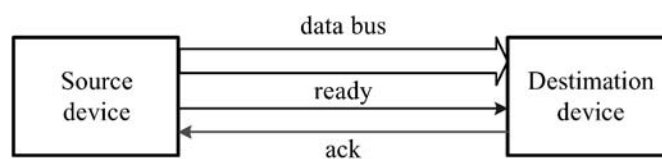
wait (count_enable) count = count - 1 ;

always

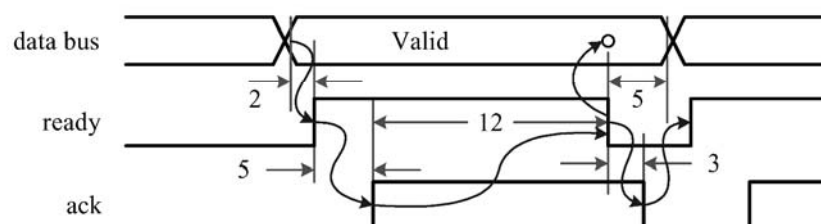
wait (!count_enable) #10 count = count - 1 ;

Level-Sensitive Event Control (cont'd)

- ❑ Used to model the handshaking operations



(a) Block diagram



(b) Timing diagram

An example illustrates the handshaking operations between two asynchronous devices.



Outline

❑ Verilog HDL

■ Overview

■ Dataflow Modeling

■ Behavior Modeling

- Procedural construct
- Procedural assignment
- Timing control
- Selection statement
- Iterative (loop) statement

■ Useful Model Technique



if...else Statement

❑ Conditional statement: if else statement

- Make a decision according to a given condition

❑ Syntax

`if (<condition>) true_statement ;`

`if (< condition >) true_statement;
else false_statement;`

`if (< condition1>) true_statement1;
else if (< condition2>) true_statement2;
else false_statement;`

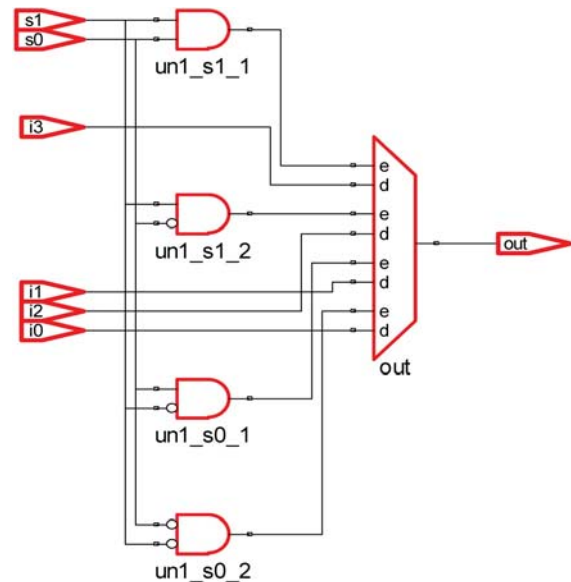
❑ condition: can be a constant or an expression

- (condition==true): has a nonzero known value
- (condition==false): has a 0, an x or a z value

Example: 4-to-1 Multiplexer

□ Using a selection statement

```
// using if...else statement
always @(*) begin
  if (s1) begin
    if (s0) out = i3;
    else   out = i2;
  end
  else begin
    if (s0) out = i1;
    else   out = i0;
  end
end
```

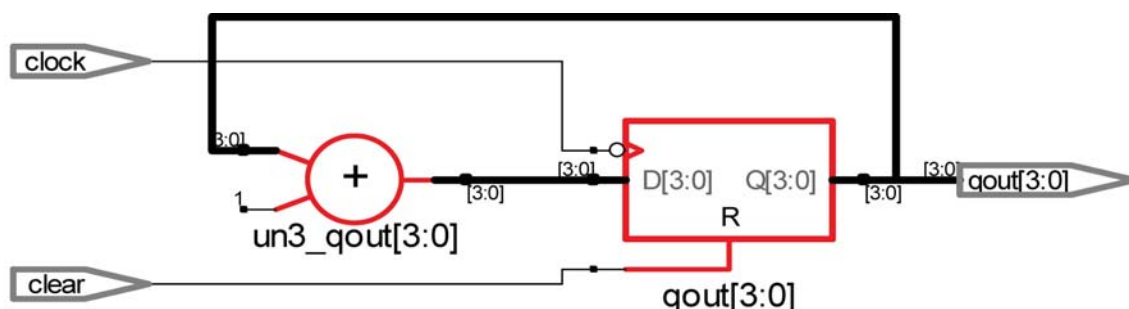


A Simple 4-bit Counter

□ 4-bit Synchronous Binary Counter with an asynchronous *clear*

// the body of the 4-bit counter.

```
always @(negedge clock or posedge clear)
  if (clear)
    qout <= 4'd0;
  else
    qout <= (qout + 1); // qout = (qout + 1) % 16;
```



case Statement

- ❑ Perform a multiway selection according to a given input condition and equivalent to a nested *if-else* statement

- ❑ Syntax

case (case_expression)

case_item1 {,case_item1}: procedural_statement1

case_item2 {,case_item2}: procedural_statement2

...

case_itemn {,case_itemn}: procedural_statementN

[**default**: procedural_statement]

endcase

- “default” is **NOT** an optional for physical design.

Example: A 4-to-1 MUX Example

```
always @(I0 or I1 or I2 or I3 or S) // this line can use
case (S)                                // always @(*)
```

```
2'b00: Y = I0;
```

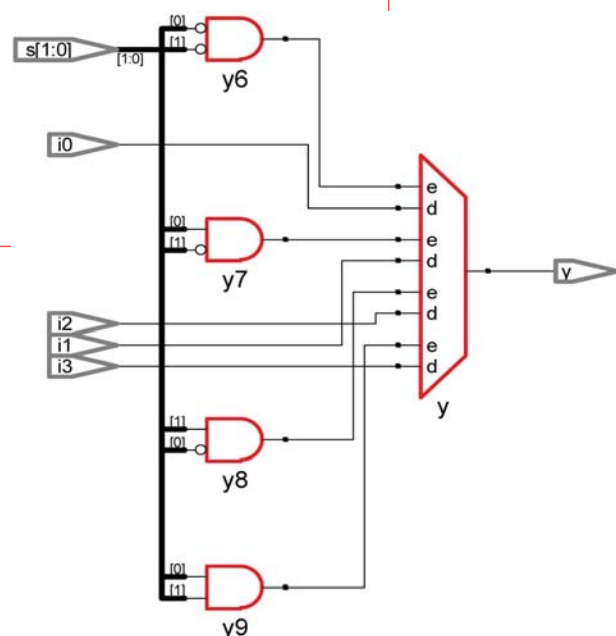
```
2'b01: Y = I1;
```

```
2'b10: Y = I2;
```

```
2'b11: Y = I3;
```

```
endcase
```

Q: How to model a 3-to-1 multiplexer?





casex and casez Statements

- ❑ Perform a multiway selection like that of the *case* statement
 - *casez* treats all *z* values as “*don’t cares*”
 - *casex* treats all *x* and *z* values as “*don’t cares*”
- ❑ *casex* and *casez* statements
 - Only compare non-*x* or *z* positions in case_expression and case_item expressions



Example: Counting the Trailing Zeros in a Nibble

```
// count the trailing zeros in a nibble
always @(data)
  casex (data)
    4'bxxx1: out = 0;
    4'bxx10: out = 1;
    4'bx100: out = 2;
    4'b1000: out = 3;
    4'b0000: out = 4;
    default: out = 3'b111;
  endcase
```

Q: Is the default statement necessary?



Outline

❑ Verilog HDL

■ Overview

■ Dataflow Modeling

■ Behavior Modeling

- Procedural construct
- Procedural assignment
- Timing control
- Selection statement
- Iterative (loop) statement

■ Useful Model Technique

Chih-Feng Wu

107



while Loop

❑ Perform a procedural statement

❑ Syntax

while (condition_expr) procedural_statement;

```
while (count < 12) count <= count + 1;
```

```
while (count <= 100 && flag) begin // complex condition and logical and (&&)
    // put statements wanted to be carried out here
end
```

Chih-Feng Wu

108

Example: Counting the Zeros in a Byte

```
// count the zeros in a byte
```

```
integer i;
```

```
always @(data) begin
```

```
    out = 0; i = 0;
```

```
    while (i <= 7) begin // simple condition
```

```
        if (data[i] == 0) out = out + 1;
```

```
        i = i + 1;
```

```
    end
```

```
end
```

```
// count the trailing zeros in a byte
```

```
integer i;
```

```
always @(data) begin
```

```
    out = 0; i = 0;
```

```
    while (data[i] == 0 && i <= 7) begin // complex condition
```

```
        out = out + 1;
```

```
        i = i + 1;
```

```
    end
```

```
end
```

Chih-Feng Wu

109

for Loop

- ☐ Perform a counting loop
- ☐ Syntax

```
for (init_expr; condition_expr; update_expr)  
    procedural_statement;
```

- The **for** loop is equivalent to the **while** loop with the following construct:

```
init_expr;  
while (condition_expr) begin  
    procedural_statement;  
    update_expr;  
end
```

Chih-Feng Wu

110

Example: Counting the Zeros in a Byte

```
// count the zeros in a byte
integer i;
always @(data) begin
    out = 0;
    for (i = 0; i <= 7; i = i + 1) // simple condition
        if (data[i] == 0)
            out = out + 1;
end
```

```
// count the trailing zeros in a byte
integer i;
always @(data) begin
    out = 0;
    for (i = 0; (data[i] == 0 && i <= 7); i = i + 1) // complex condition
        out = out + 1;
end
```

repeat Loop

- ❑ Perform a procedural statement with a specified number of times

- ❑ Syntax

repeat (**counter_expr**) procedural_statement;

```
i = 0;
repeat (32) begin
    state[i] = 0;    // initialize to zero
    i = i + 1;       // next time
end

repeat (cycles) begin                // cycle must be evaluated to
                                     // a number before entering

    @(posedge clock) buffer[i] <= data;
    i <= i + 1;
end
```



forever Loop

- ❑ Perform a procedural statement until the \$finish system task is encountered.
- ❑ Syntax

forever procedural_statement;

```
initial begin
```

```
    clock <= 0;
```

```
    forever begin
```

```
        #10 clock <= 1;
```

```
        #5  clock <= 0;
```

```
    end
```

```
end
```

```
reg clock, x, y;
```

```
initial
```

```
    forever @(posedge clock) x <= y;
```



Outline

❑ Verilog HDL

■ Overview

■ Dataflow Modeling

■ Behavior Modeling

- Procedural construct
- Procedural assignment
- Timing control
- Selection statement
- Iterative (loop) statement

■ Useful Model Technique



Compiler Directive

- ❑ `<keyword>`
 - ``define` and ``undef`
 - ``include`
 - ``ifdef`, ``else`, ``elsif`, ``endif` and ``ifndef`
 - ``timescale`
 - ``celldefine` and ``endcelldefine`
 - ``line`
 - ``default_nettype`
 - ``unconnected_drive` and ``nounconnected_drive`
 - ``resetall`



``define` and ``undef`

- ❑ ``define`: create a macro for text substitution
- ❑ Syntax: ``define macro_name [(formal_arguments)] macro_text`
``macro_name [(actual_argument1, ..., actual_argumentn)]`
 For example:
 1. ``define BUS_WIDTH 8`
``BUS_WIDTH` will be substituted by 8 in Verilog RTL.
 2. ``define min(a, b) ((a) < (b) ? (a) : (b))`

```

y = `min(p+q, r+s);           // macro call
y = ((p+q) < (r+s) ? (p+q) : (r+s));

```
- ❑ ``undef`: remove a previously defined text macro
- ❑ Syntax: ``undef macro_name`



`include

- ❑ **`include**: insert an entire source file into the main program
 - Similar to that of **#include** in C-program
 - Syntax: **`include** “filename”
- For example: **`include** “count.v”
- `include** “../program/chapter06/count.v”



`ifdef, `else, `elseif, `endif and `ifndef

- ❑ Optional parts of a source file can be compiled conditionally based on some macro definitions.
- ❑ Two types of conditionally compiled directives
 - Check the existence of a macro definition: **`ifdef**
 - Check the lack of a macro definition: **`ifndef**
 - Syntax:

```
`ifdef macro_name
    ifdef_group_of_lines

{elseif macro_name
    elseif_group_of_lines}

[else else_group_of_lines]

`endif
```

Example:

```
`define DWORD
:
:
`ifdef DWORD
    parameter WORD_Size=32;
`else
    parameter WORD_Size=16;
`endif
```



`ifdef, `else, `elsif, `endif and `ifndef (cont'd)

❑ **`ifndef**

■ Syntax:

```
`ifndef macro_name
    ifdef_group_of_lines

{`elsif macro_name elsif_group_of_lines}

[`else else_group_of_lines]

`endif
```

```
module xor_op(a, b, c);
    output a;
    input b, c;

    `ifndef DataFlow
        xor a1 (a, b, c);
    `else
        wire a = b ^ c;
    `endif
endmodule
```



`timescale

❑ Specify both the physical unit of measure or the time scale of numerical time delay and the resolution of the time scale

■ Syntax: **`timescale time_unit/time_precision**

■ For example:

```
`timescale 10ns /1ns
#2.55 a = b + 1; // 26 ns
#2.54 b = c + 1; // 25 ns
```

```
`timescale 1ns /10ps
#2.55 a = b + 1; // 2.55 ns
#2.54 b = c + 1; // 2.54 ns
```

File I/O

❑ Opening and closing file system

- `$fopen(file_name, mode);`

- `$fclose(fd);`

- **mode:**

1. “**r**” or “**rb**”: open file for **reading** at the beginning of file. Return 0 if it fails.
2. “**w**” or “**wb**”: open file for **writing** at the beginning of file or create file if it does not exist
3. “**a**” or “**ab**”: open file for **appending** at the beginning of file or create file if it does not exist

❑ File output system

- `$fdisplay, $fdisplayb, $fdisplayh, $fdisplayo`

- `$fwrite, $fwriteb, $fwriteh, $fwriteo`

- `$fmonitor, $fmonitorb, $fmonitorh, $fmonitorto`

- `$fstorbe, $fstorbeb, $fstorbeh, $fstorbeo`

Chih-Feng Wu

121

```
module fact_file;
```

```
// test the fact function
```

```
integer n, result, fd;
```

```
initial
```

```
begin
```

```
    fd = $fopen("fact_result.dat", "w");
```

```
    if (fd == 0)
```

```
        $display("File open error !");
```

```
    else
```

```
        begin
```

```
            for (n = 0; n <= 12; n = n + 1) begin
```

```
                result = fact(n);
```

```
                $fdisplay (fd, "%0d factorial = %0d", n, result);
```

```
            end
```

```
            $fclose (fd);
```

```
        end
```

```
end
```

```
// define fact function
```

```
function automatic integer fact;
```

```
input [31:0] n;
```

```
// the body of function
```

```
    if (n == 0) fact = 1;
```

```
    else fact = n * fact(n - 1);
```

```
endfunction
```

```
endmodule
```

122

File I/O (cont'd)

❑ File input system

- \$fgetc, \$ungetc, \$fgets, \$fscanf
- \$fread, **\$readmemb**, **\$readmemh**, \$ftell
- \$fseek, \$rewind, \$ferror

❑ Reading binary file

1. code=\$fread(mem, fd, [start], [count]);
2. code=\$fread(reg, fd);
3. \$readmemb(**b**("<file_name>", <memory_name>, [start], [end]));
4. \$readmem**h**("<file_name>", <memory_name>, [start], [end]);

Example:

```
reg [7:0] cache_mem[255:0];
```

```
initial $readmemb("cache_mem.dat", cache_mem);           // start at 0
```

```
initial $readmemb("cache_mem.dat", cache_mem, 24);        // start at 24
```

```
initial $readmemb("cache_mem.dat", cache_mem, 156, 1);    // start at 156
```